

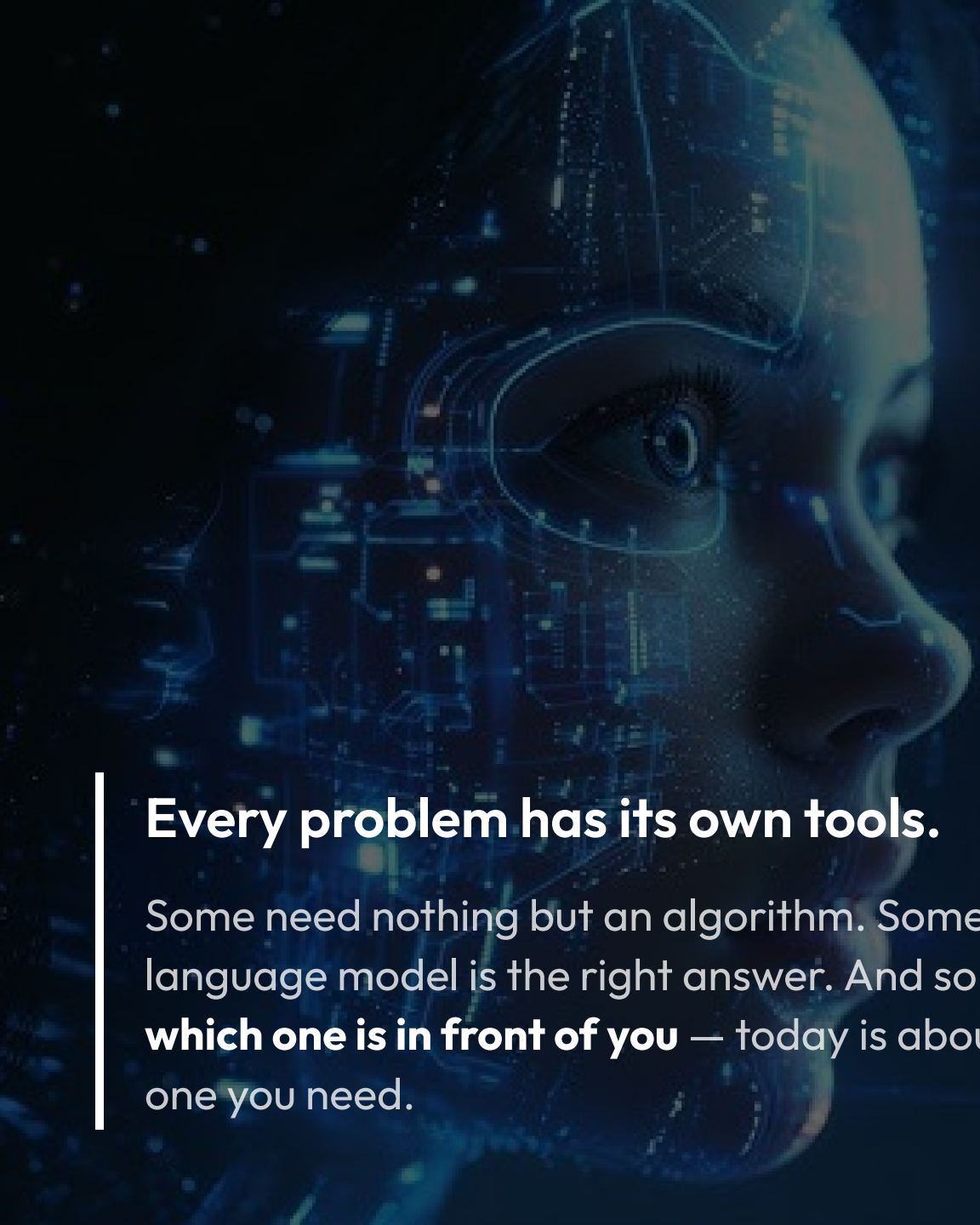
Introduction to Agents

LLM-Agents

Presented by: **Aleksandr Algazinov**



HSE University



AI AGENTS

Every problem has its own tools.

Some need nothing but an algorithm. Some need classical machine learning. For some, one call to a language model is the right answer. And some need an agent system. The engineering is knowing **which one is in front of you** — today is about the last of the four, and about how often it is not the one you need.

Motivation

You can already call a model. Now consider a task it cannot do in one call.

"WHAT DID OUR COMPETITOR ANNOUNCE THIS WEEK, AND HOW DOES IT COMPARE TO OUR ROADMAP?"

Needs a lookup it does not have, a document it has not read, and a decision about what to do *after* reading it.

ONE CALL

Produces a fluent answer from whatever was in the weights. Possibly invented.

A LOOP AROUND THE SAME MODEL

Can look, read, and decide again with what it found.

That loop is the whole subject of the next ten weeks — and the reason to be sceptical of it.

And knowing how it works buys you three things

WHEN NOT TO BUILD ONE

Many tasks that look agentic are not.

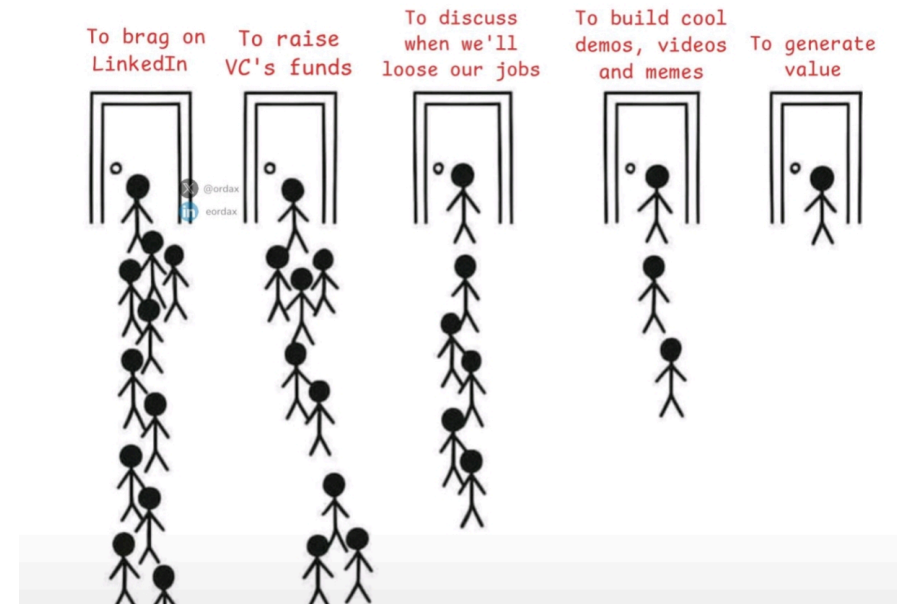
WHY IT BROKE

Schema, termination, context or model — you can only tell if you know the parts.

WHAT IT WILL COST

The loop's shape is the call count — your bill and your latency.

How people is using Agentic AI

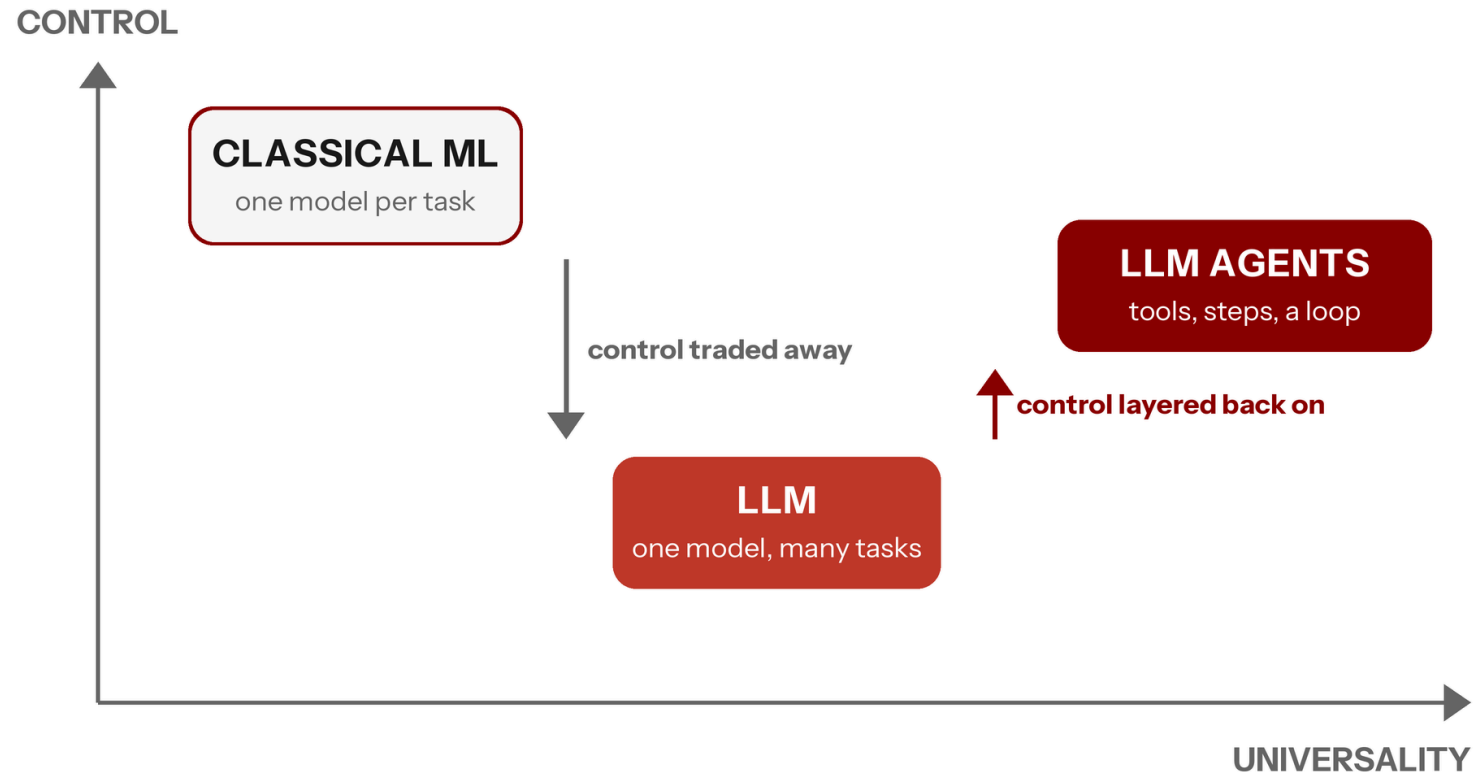


Control or Universality

Ten years of applied AI, as one running trade-off

The trade-off everything else follows from

Every paradigm shift of the last decade is a move on these two axes. Understanding the trade explains **why each shift happened** — and why none of them deleted the one before it.



Act I • Classical ML — full control, task by task

WHAT YOU DID

Five to ten years ago, building an AI system meant starting almost from scratch for each new problem. Data was prepared in a task-specific format, and real effort went into selecting and tuning the right architecture for *that* task.

WHAT IT COST

It worked, and it was expensive. Every new task meant a new pipeline, new experiments, new decisions — all from the ground up. Nothing you built for one problem transferred to the next.

WHAT YOU HAD IN EXCHANGE

Complete control. You chose the features, the objective and the loss. When it was wrong, there was a specific place to go and change something.

Act II · LLMs — universality, at a price

WHAT CHANGED

After the transformer, generative models arrived. Instead of designing an architecture per task, you could adapt a pre-trained model: fine-tune it, prompt it, and have something working in a fraction of the time. **One model, many tasks.**

WHAT WAS LOST

When you push all your data into a language model and hope for the best, you have far fewer places to intervene. The precision and flexibility that made classical ML powerful were partly gone.

THE SHAPE OF THE TRADE

You bought range with control. That is a genuinely good trade for most problems — and a bad one for any problem where being wrong is expensive and you need to know why.

Act III · Agents — getting the control back

AGENTS WERE THE ANSWER TO THE CONTROL LOSS

They are still LLM-based. Instead of predicting the next token and stopping, they operate with more machinery: task planning and decomposition, integration with external tools, multi-step reasoning.

WHAT THAT BUYS

The universality of an LLM, with a new layer of control on top — you choose the tools, the steps, and when it stops.

WHAT IT DOES NOT BUY

The control of classical ML. You are steering a system whose core step is still a next-token prediction you did not write.

Nothing was replaced

- 1 Classical ML is **still the right choice** for a great many problems. It did not become obsolete when GPT shipped.
- 2 A single LLM call is still the right choice for more problems than most people building agents would like to admit.
- 3 Each new paradigm **adds a level of abstraction while trying to recover what the previous one sacrificed**. That is the whole pattern.

A FRAMING, NOT A FINDING

This arc is a way of organising ten years, not a result anybody measured. It is useful because it predicts what the *next* shift will be about.

Which one does the problem need?

	Classical ML	One LLM call	An agent
You control	features, objective, loss	the prompt, and little else	tools, steps, stop conditions
It costs you	a pipeline per task	almost nothing per call	latency, money, predictability
Reach for it when	the task is narrow, repeated, and measurable	the answer is in the prompt already	the next step depends on what was found
You know it works by	a held-out metric	reading the output	...this is the hard part — Lecture 13

The last row is the one nobody plans for, and it is why the bottom of this table is harder to live with than the top.

DISCUSSION

Three problems. Which of the three needs classical ML, which needs one LLM call, and which needs an agent?

A. Flag the 2% of card transactions most likely to be fraud, on 40 million rows a day. · **B.** Turn a customer's angry review into a polite two-sentence reply. · **C.** A customer says an order never arrived — find out what happened and make it right.

Three problems, three different answers

A • FRAUD ON 40M ROWS — CLASSICAL ML

Narrow, repeated, and it has a label. You need calibrated probabilities, a threshold you can tune, and an answer in milliseconds. A language model is worse at this *and* far more expensive.

B • THE POLITE REPLY — ONE LLM CALL

Everything needed is already in the prompt. No lookup, no side effect, no decision about what comes next. Wrapping this in an agent adds latency and failure modes and buys nothing.

C • THE MISSING ORDER — AN AGENT

Check the courier, check the warehouse, check the payment. **What you check second depends on what the first check said** — and that dependency is the thing an agent is for.

The rule underneath

TREAT EACH PROBLEM INDIVIDUALLY

There is no default architecture. The question is never "should we use agents this year" — it is what this particular problem needs, and sometimes the honest answer contains no model at all.

THE DIRECTION OF TRAVEL IS REAL

Each paradigm did genuinely widen what is buildable. Agents are not hype-only.

AND IT IS NOT A RANKING

"Newer" is not "better for your problem". A gradient-boosted tree from 2016 still wins a great many production decisions.

What Is an Agent?

The question predates LLMs by decades

The definition

AN AGENT

Something that **perceives** an environment, **decides**, and **acts** on it — in pursuit of a goal.

NOTE WHAT IS ABSENT

Nothing about language, learning, or intelligence. A thermostat qualifies.

THE GOAL IS DOING THE WORK

A model that answers is judged on its answer. An agent is judged on whether the task got done.

Autonomy is a scale

SUGGESTS

Proposes; a human does the work. Safest, slowest, and often the right first version.

ACTS WITH APPROVAL

Proposes a concrete action and waits for yes. Where most useful systems actually sit.

ACTS ALONE

Decides and executes. Reserve for actions that are cheap, reversible, or both.

You choose this per **action**, not per system. The same agent can send a draft freely and require approval to send an invoice.

Remember this shape. It comes back twice more today.

Two older kinds

RULE-BASED / REFLEX

- Condition → action, written by a human
- Predictable and auditable
- Brittle outside the cases the author imagined

REINFORCEMENT LEARNING

- A policy learned from reward
- Handles situations nobody enumerated
- Needs an environment and a lot of trials

We are not teaching RL. The point is only that **agents are an old idea**, and LLMs are a new way to build the deciding part.

Reactive or deliberative

REACTIVE

Act on what you see right now. Fast, cheap, and carrying no memory of why it did the last thing.

REACTIVE, CONCRETELY

A spam filter. Each message is judged on what it contains, nothing is remembered between messages, and the decision is immediate.

DELIBERATIVE

Build a model of the situation and plan before acting. Slower, and able to handle steps that only pay off later.

DELIBERATIVE, CONCRETELY

A route planner. It considers journeys it will not take in order to choose the one it will, and the first move only makes sense given the rest.

Both come back today under names you will recognise: **ReAct** and **Plan-and-Execute**. The forty-year-old distinction is still the distinction.

So what is an *LLM* agent?

THIS COURSE'S WORKING DEFINITION

An LLM agent is an **extension of a plain model call**. Invoke a model and take the answer — that is an LLM application. Change the system prompt, hand it a tool, give it skills, wrap a loop around it, or set several of them working on each other: **anything beyond the bare invoke**, and you are building an agent.

THIS IS NOT THE ONLY DEFINITION

Plenty of practitioners reserve the word for tool calling, multi-step reasoning or dynamic planning. That is a legitimate line to draw, and a common one.

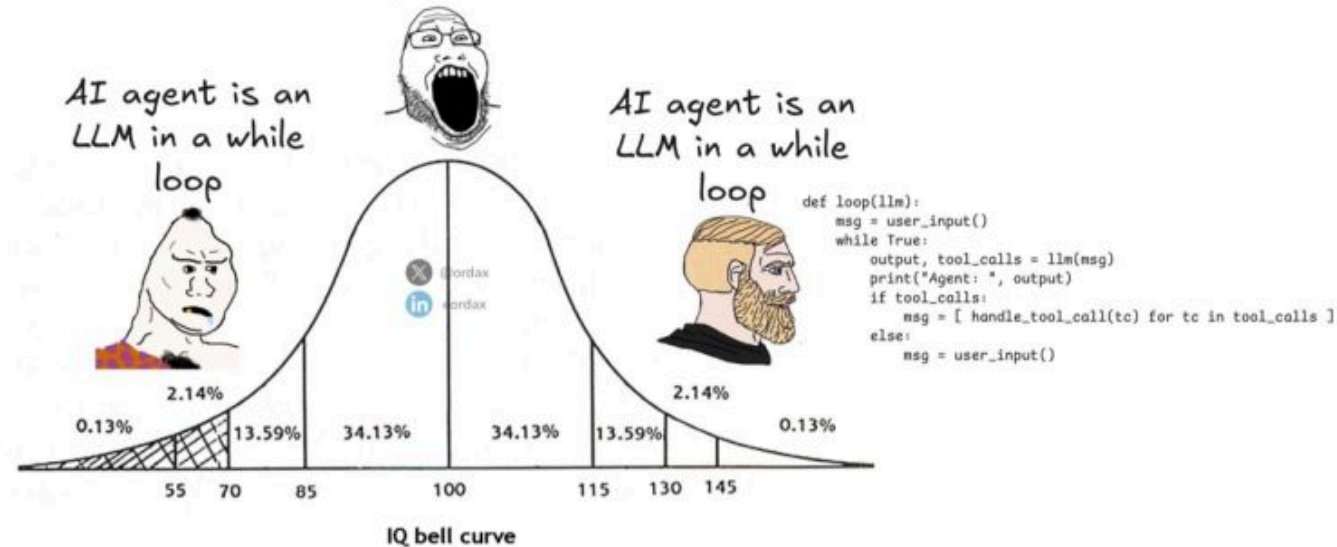
DRAW YOURS WHEREVER YOU LIKE

The only requirement is that you can say *where* it is and *why*. Arguing about the word is how teams waste a sprint.

What do they mean by "AI agent"?

What do they mean by AI Agent?

An AI agent is a software system that autonomously perceives its environment, makes decisions, and takes actions to achieve specific goals, often learning and adapting over time



A scale, not a switch

Because the definition is about **degree**, the useful question is never *is this an agent*. It is: how far along do I need to be, and does this task pay for it?

AN LLM APPLICATION

A BARE CALL

llm(prompt), and you are done

A SYSTEM PROMPT

a role, rules, a format

THIS COURSE

an agent starts here

TOOLS

now it can look things up

A COMMON STRICTER LINE

an agent starts here

A LOOP

it decides whether to go again

SEVERAL AGENTS

they check each other's work

AN AGENT SYSTEM

From a Call to an Agent

Six examples, one thing added each time

Example 1 · A plain call

```
import os
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv() # API_KEY, BASE_URL, MODEL_NAME
MODEL = os.environ["MODEL_NAME"]
client = OpenAI(api_key=os.environ["API_KEY"],
                base_url=os.environ["BASE_URL"])

def llm(prompt: str, system: str = "") -> str:
    msgs = [{"role": "system", "content": system}] if system else []
    msgs.append({"role": "user", "content": prompt})
    r = client.chat.completions.create(model=MODEL, messages=msgs)
    return r.choices[0].message.content

idea = "an AI that decides whose turn it is to do the dishes"
answer = llm("Give me five startup names for " + idea)
```

This is an LLM application. One call in, one answer out. Nothing decides what to do next, because there is no "next" — and every snippet that follows still calls this `llm()`.

Example 2 · The same call, with a system prompt

```
def persona(system: str):  
    def wrap(fn):  
        def inner(*args, **kwargs):  
            return llm(fn(*args, **kwargs), system=system)  
        return inner  
    return wrap  
  
@persona("You are a VC who has seen ten thousand pitch decks. "  
        "Be blunt. One line each, and name the moat if there is one.")  
def judge(names: str) -> str:  
    return "Rate these startup names out of ten:\n" + names
```

Same model, same API call, **different behaviour** — and you did not retrain anything. By this course's definition, you have now built an agent.

Example 3 · Give it one tool

```
import requests

@tool
def is_taken(domain: str) -> str:
    """Say whether a domain already has a live website."""
    try:
        requests.get(f"https://{domain}", timeout=5)
        return f"{domain} is taken."
    except requests.RequestException:
        return f"{domain} looks free."

run(f"Pick the best name for {idea}, then check the .com.",
    tools=[is_taken], system="You are a blunt VC.")

# @tool and run() are not library code – they are the next two slides.
```

It can now find out things that were not in the prompt — a real capability change, not a phrasing one. And note what had to go: **llm()** is gone, because a helper that returns a string cannot show you a tool call.

Example 4 · What `@tool` actually is

```
import inspect, json

def tool(fn):
    names = list(inspect.signature(fn).parameters)
    fn.schema = {"type": "function", "function": {
        "name": fn.__name__, "description": fn.__doc__,
        "parameters": {"type": "object", "required": names,
            "properties": {n: {"type": "string"} for n in names}}}}
    return fn
```

A decorator that reads the function's own signature and docstring, and hangs a JSON description of it on the function. **The model never sees your code** — it sees this dictionary.

Every parameter is typed `string` here. Doing it properly is **Lecture 05**.

Example 5 • What `run()` actually is

```
def run(task: str, tools=(), system="") -> str:
    fns = {t.__name__: t for t in tools}
    msgs = [{"role": "system", "content": system},
            {"role": "user", "content": task}]
    while True:
        m = client.chat.completions.create(model=MODEL, messages=msgs,
                                           tools=[t.schema for t in tools] or None).choices[0].message
        msgs.append(m)
        if not m.tool_calls:
            return m.content
        for c in m.tool_calls:
            out = fns[c.function.name](**json.loads(c.function.arguments))
            msgs.append({"role": "tool", "tool_call_id": c.id,
                        "content": out})
```

That loop is the whole idea. Call; if the reply asks for a tool, run it, append the result, call again.

A function, not a class — **fourteen lines, and you can read all of them.** Everything that makes it *correct* rather than merely working is **Lecture 05**.

Example 6 · Add a second agent

```
NAMER = "You name startups. Three options, no explanation."
VC     = "You are a VC who has seen ten thousand pitch decks. Be blunt."

names = run(f"Name a startup: {idea}", system=NAMER)
notes = run(f"Rate these, and check whether the .com is free:\n{names}",
            tools=[is_taken], system=VC)
final = run(f"Name a startup: {idea}\n\nFix these:\n{notes}", system=NAMER)

# Who talks to whom, when they stop, and what it costs
# to run two context windows                                -> Lecture 08
```

Three calls instead of one. Whether the output is three times better is a separate question — often, no.

YOU HAVE JUST WRITTEN A PATTERN WITH A NAME

Generate, critique, revise. You meet it later today as **reflection** — and why it often fails.

What each example actually bought

Example	What was added	What it cost	Owned by
A plain call	—	one call	Lecture 02
A system prompt	behaviour you specify	nothing	today
A tool	reach beyond the prompt	a schema, a loop, a failure mode	Lecture 05
A second agent	a check on the first	3× the calls, 2× the context	Lecture 08

All of it is ordinary Python. **Nowhere in those six examples does something magical happen** — which is exactly why the word "agent" is argued about.

DISCUSSION

Pull five fields out of 10,000 invoices and write them to a database.

Would you build that as an agent? Argue both sides — and name something an agent costs you that a plain script does not.

Three answers, and only one of them is an agent

IF THE INVOICES SHARE A LAYOUT — A SCRIPT

Fixed fields at fixed positions. No model in the loop at all, and the same answer every time you run it.

IF THE LAYOUTS VARY — ONE MODEL CALL PER INVOICE

Extraction needs judgement, so a model earns its place. But nothing decides *what to do next* — no loop, no agent.

AN AGENT ONLY IF THE NEXT STEP DEPENDS ON WHAT IT FOUND

Chase a missing PO number, query a second system, escalate a mismatch. Flexibility is the only thing worth paying for.

When you do not need an agent

USE CODE INSTEAD

- The task is deterministic
- The steps never change
- You need the same answer every time

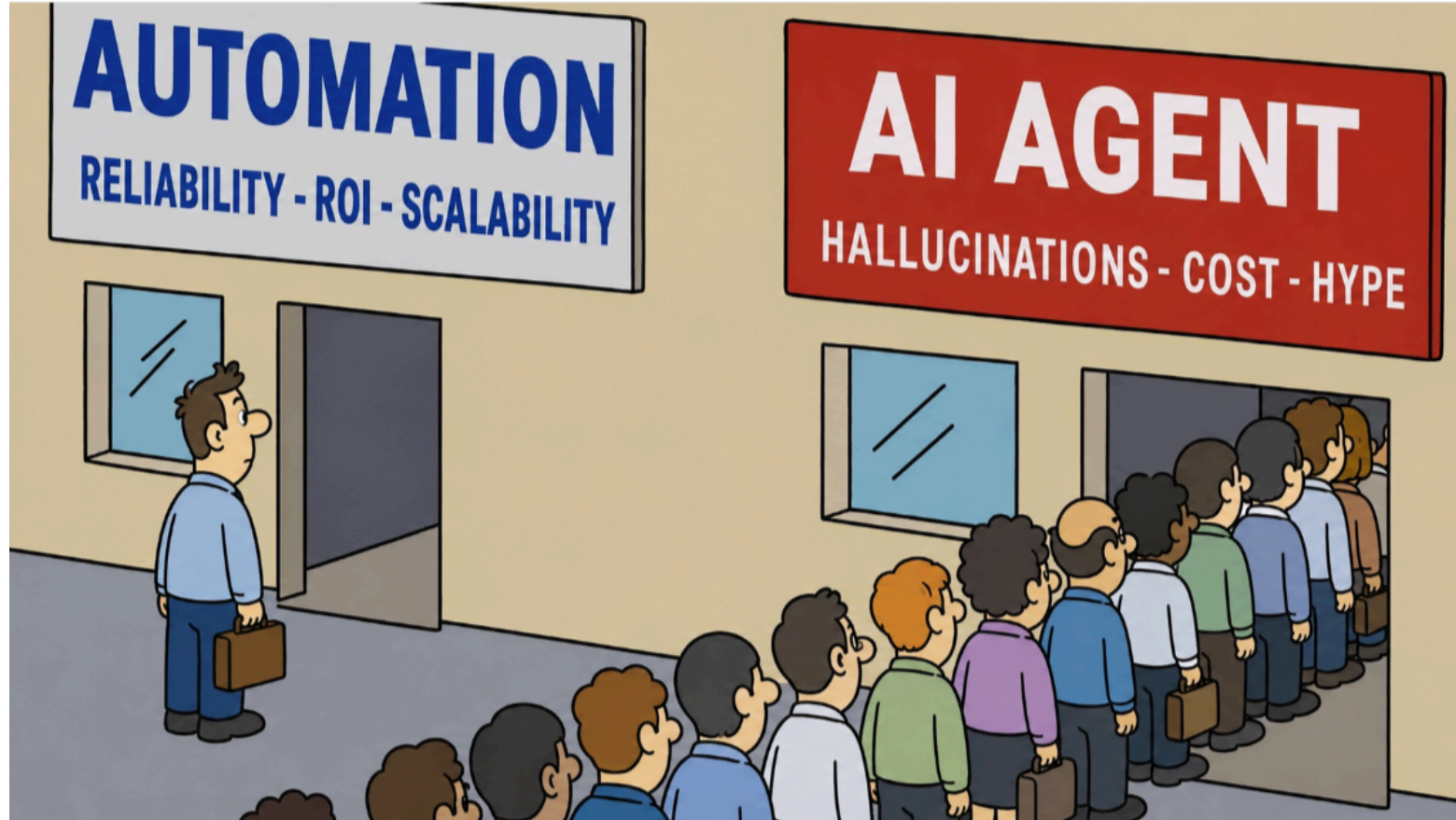
USE ONE CALL INSTEAD

- Everything needed is already in the prompt
- No lookup, no side effect
- One output, no decision about what is next

WHAT AN AGENT ACTUALLY COSTS YOU

Predictability, latency, and money. Anthropic puts it plainly: agentic systems *trade latency and cost for better task performance*. If you are not using the flexibility, you are only paying.

The queue nobody wants to join

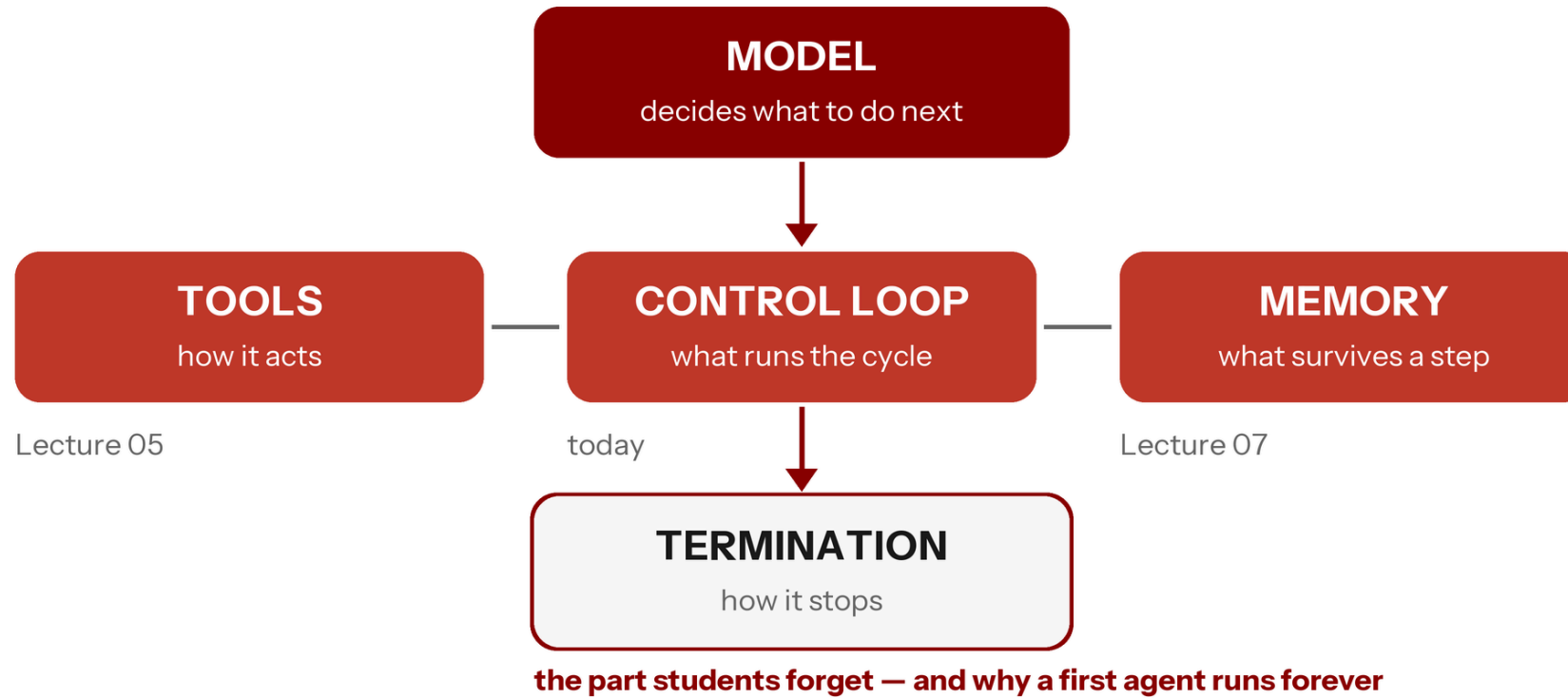


Anatomy of an Agent

Five parts, four of them taught later

The anatomy

Five parts. Four of them get their own lecture later — which is one way of reading the syllabus.



an agent is a model plus the four things around it

Termination: the part you have to write

NOTHING STOPS AN AGENT BY DEFAULT

The loop runs until something says stop. If you did not write that something, the answer is "your budget".

CONDITIONS WORTH HAVING

- The model answered instead of acting
- A step budget was reached
- A cost or time ceiling was hit
- The same action repeated n times

WHAT EACH ONE IS FOR

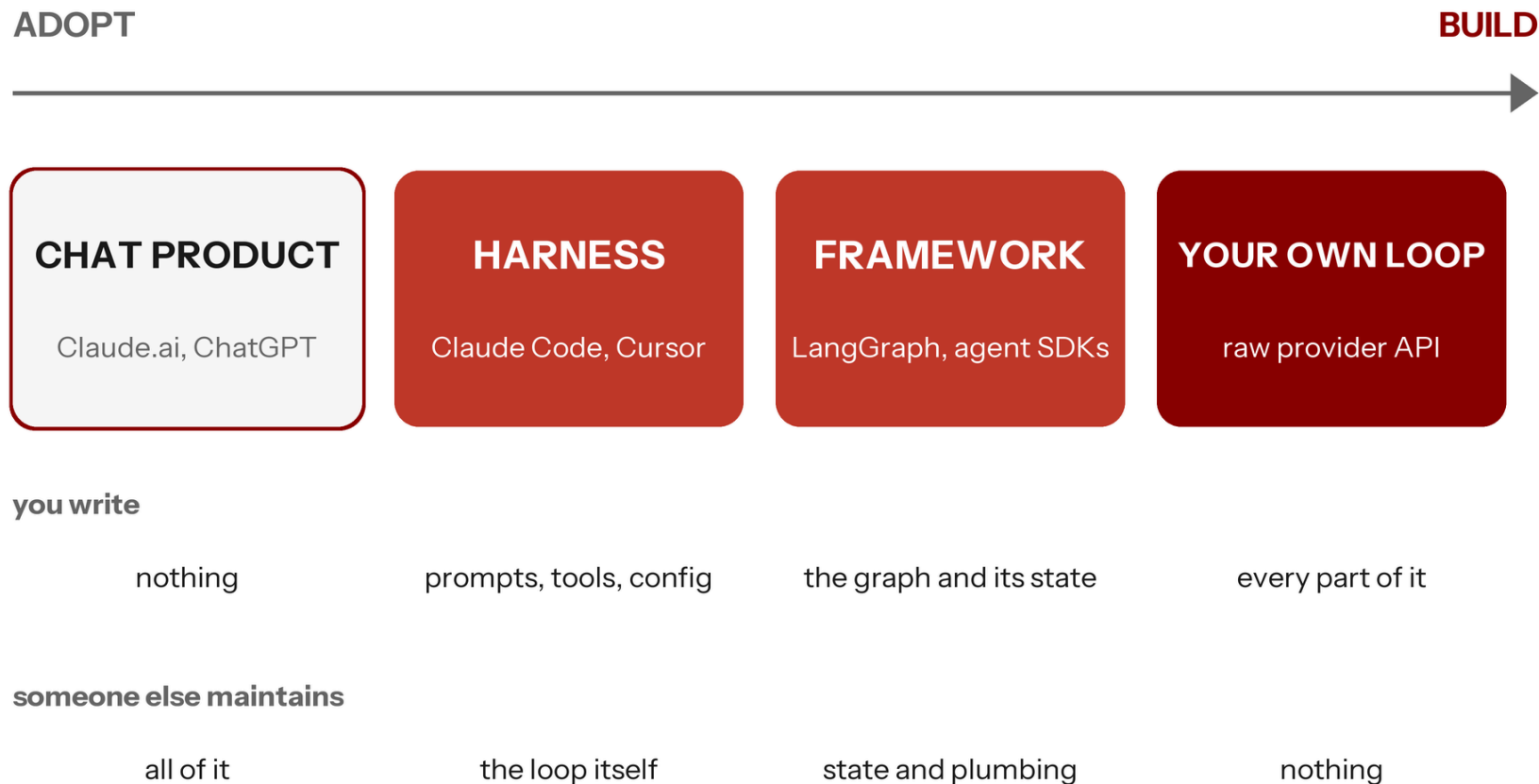
- The first is success
- The rest are all failure
- Log *which* one fired — it is your best debugging signal

Build or Adopt?

You already use one. Why write your own?

Ownership is a scale

Four rungs, not two options — and like autonomy, and like how far past a bare call you go, you pick per system rather than once and forever. **That is the third scale today.**



What owning the loop buys you

CONTROL

You choose the tools, the stop conditions and the retry policy — a narrow repeatable path, where a harness offers an open-ended one.

DATA BOUNDARY

You decide what the agent may touch. Scoping tools, files and network shrinks the blast radius; only self-hosting removes the vendor (Lecture 03).

INTEGRATION

It runs *inside* your system rather than beside it — your queue, your database, your permissions.

COST AND PROOF

You control how many steps run, and what each one is allowed to cost. A loop you instrumented is one you can regression-test (Lecture 13).

The one nobody expects

The task: triage every support ticket the moment it arrives — including 03:00 on a Sunday.

WHAT CLAUDE CODE NEEDS TO RUN

A terminal someone opened. A question someone typed. An answer someone is waiting to read. A permission prompt someone can approve. **Every one of those is a person.**

WHAT THE 03:00 TICKET HAS

A queue message, a five-minute SLA and a database transaction. No terminal. Nobody typed the question, and nobody is awake to approve anything.

WHY THIS IS USUALLY THE ONE THAT DECIDES IT

You do not write your own loop because the harness is bad — it is excellent, at what it is for. You write it because **there is no person for the harness to talk to**. Control and cost are arguments you can lose on the numbers; this one is a fact about where your code has to run, and it ends the conversation.

When not to build your own agent

ADOPT INSTEAD WHEN

- The work is exploratory
- It runs once, or rarely
- Its shape changes every week
- You cannot yet tell whether a change helped

WHAT BUILDING ACTUALLY COSTS

- Retries, timeouts, budgets
- Context management
- Sandboxing and approvals
- Logs and traces
- All of it, maintained forever, by you

BE HONEST ABOUT THE CROSSOVER

Anthropic's own guidance is to add agentic complexity *only when simpler solutions fall short*. You are not choosing between good and bad — you are choosing what you are willing to maintain.

Where this course puts you

Rung	Where you meet it
Your own loop, on raw provider SDKs	Lecture 05 — tools, schemas, validation, retries
Frameworks that pre-build the loop	Lecture 09 — LangGraph and the alternatives
Someone else's harness, extended	Lecture 11 — agentic coding in practice
Whether any of it actually works	Lecture 13 — evaluation and operations

You will stand on every rung before this course ends. Today only fixes the question you ask when you choose one.

THE PRINCIPLE UNDERNEATH

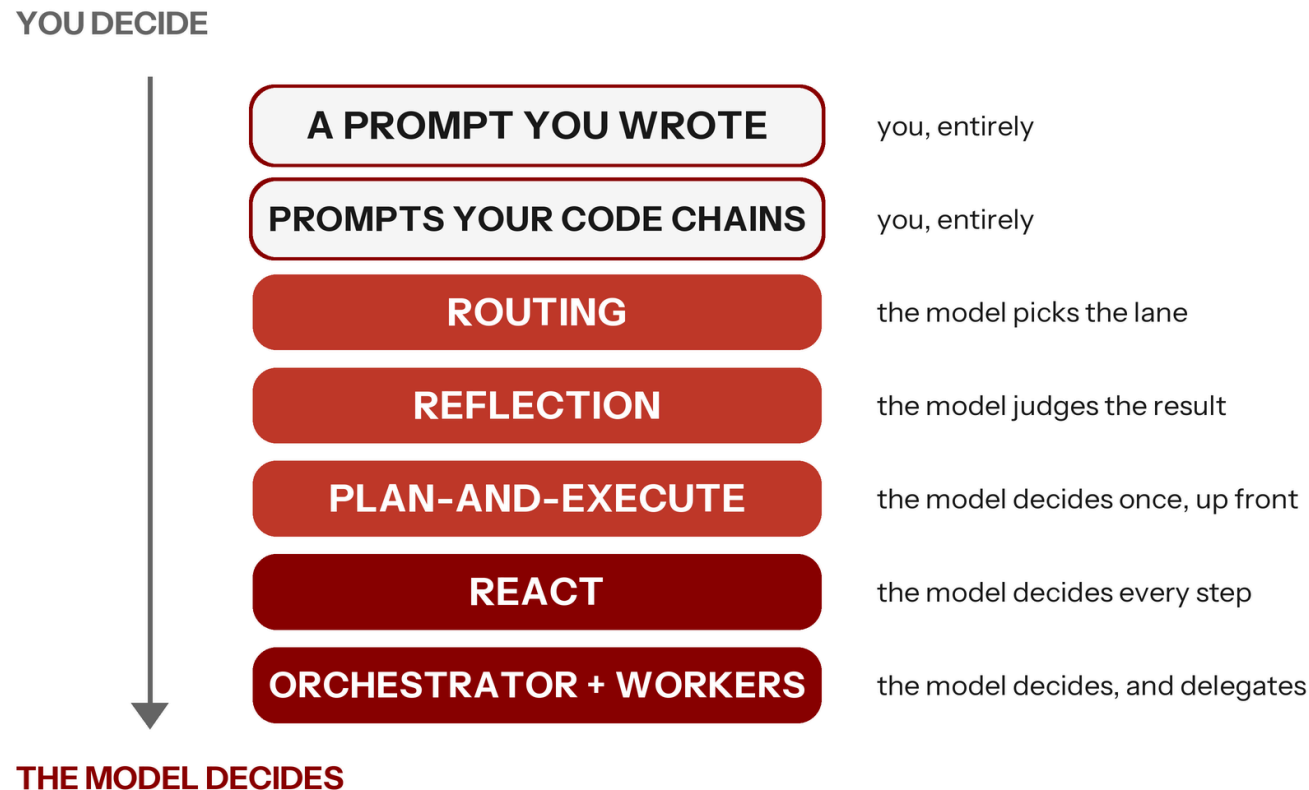
Practitioners state it as three rules — own your prompts, own your context window, own your control flow. Every rung above is a decision about how many of the three you keep. *12-Factor Agents*

Who Decides What Happens Next?

One axis, and the running order for the rest of today

The lineage

Not a chronology — **one axis**. Each shape hands the model a little more of the control flow. That is the evolution, and it is also the order of the next six slides.



Workflow, or self-directed?

A WORKFLOW

LLMs and tools orchestrated through **predefined code paths**. You wrote the path. The model fills in the boxes.

SELF-DIRECTED

The model **dynamically directs its own process** and tool usage, keeping control over how it accomplishes the task.

Anthropic call the second one *an agent*, and so does most of the field. Note the word, though: **by this lecture's definition both sides are already agents**. Same word, narrower sense. The only question here is *who picks the next step*.

FOUR OF THE SIX SHAPES ARE WORKFLOWS

Routing, reflection, reflexion, orchestrator-worker. The path exists before the model runs.

AND MOST SHIPPED SYSTEMS ARE WORKFLOWS

That is not a criticism. A predefined path is testable, cheaper, and easier to explain to whoever has to sign it off.

Routing

Classify the input, then send it down the right branch.

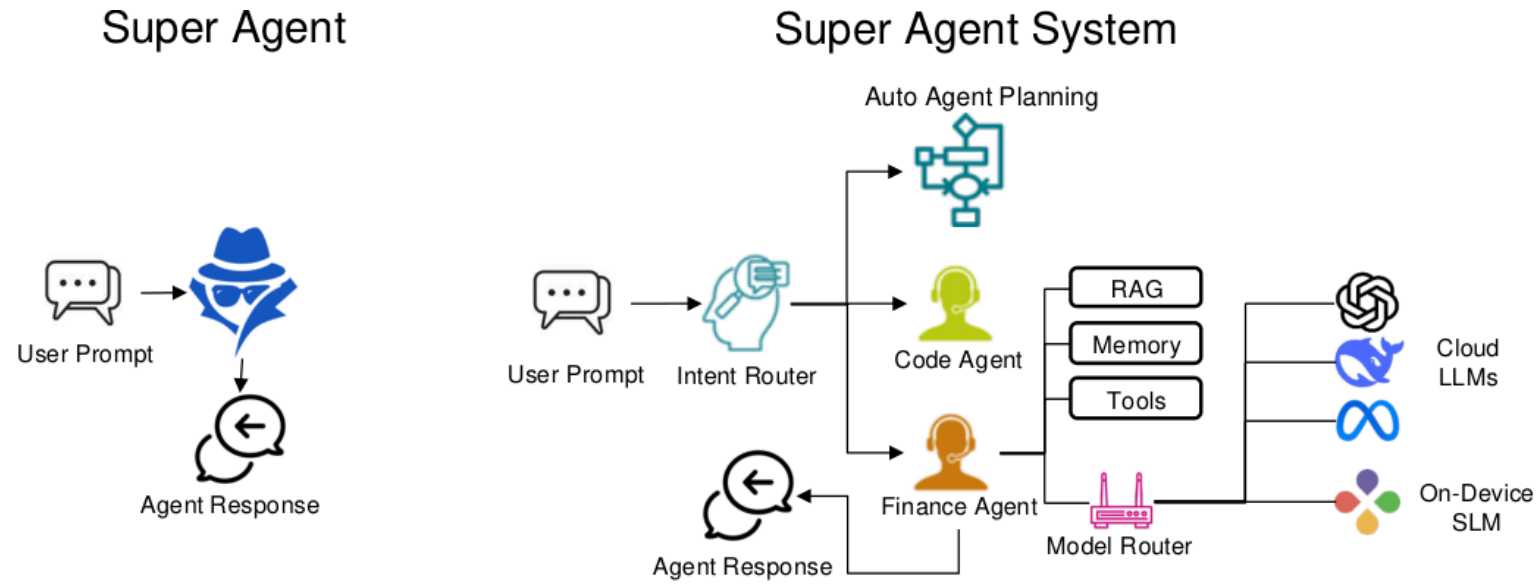


Figure: Yu et al. 2025. **The intent router on the left is today's.** The *model* router on the right — a cheaper or dearer model per task — is Lecture 08.

When routing wins, and how it fails

REACH FOR IT WHEN

Inputs fall into a few recognisable kinds, and handling each kind differently beats handling them all the same way. A refund request and a technical question need different prompts, different tools, and possibly different people.

ITS CHARACTERISTIC FAILURE

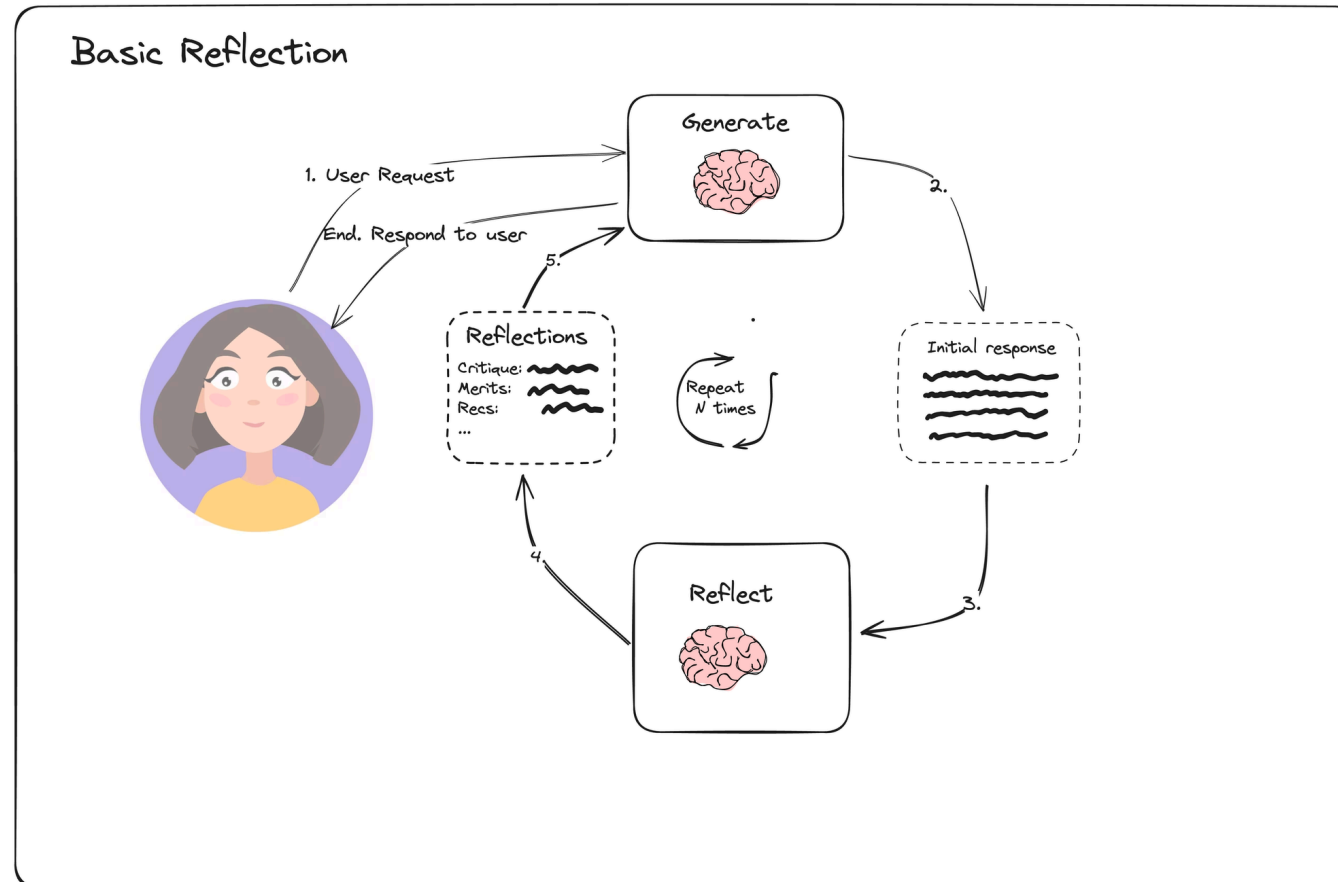
The classifier is wrong and **nothing tells you**. The wrong branch still produces fluent, confident output, so the error surfaces as a quality complaint weeks later rather than as an exception.

THE MITIGATION IS BORING AND IT WORKS

Log the branch taken alongside the input. A routing mistake is trivially visible in a table of inputs and chosen branches, and invisible everywhere else.

Reflection

Generate, critique, revise — and repeat a bounded number of times. Anthropic calls it evaluator-optimizer: one call produces a response, another evaluates it and feeds back.



DISCUSSION

You ask a model to review its own answer and fix what is wrong with it. Does the answer get better?

Assume no tests, no search, no second model, no human — just the model, its own output, and "check your work". Argue it.

Mostly not — and that result is the design rule

THE FINDING

On reasoning tasks, models largely **cannot** self-correct from their own inherent capabilities alone, and performance sometimes *degrades* after self-correction. Huang et al., ICLR 2024.

WHY IT GOES WRONG

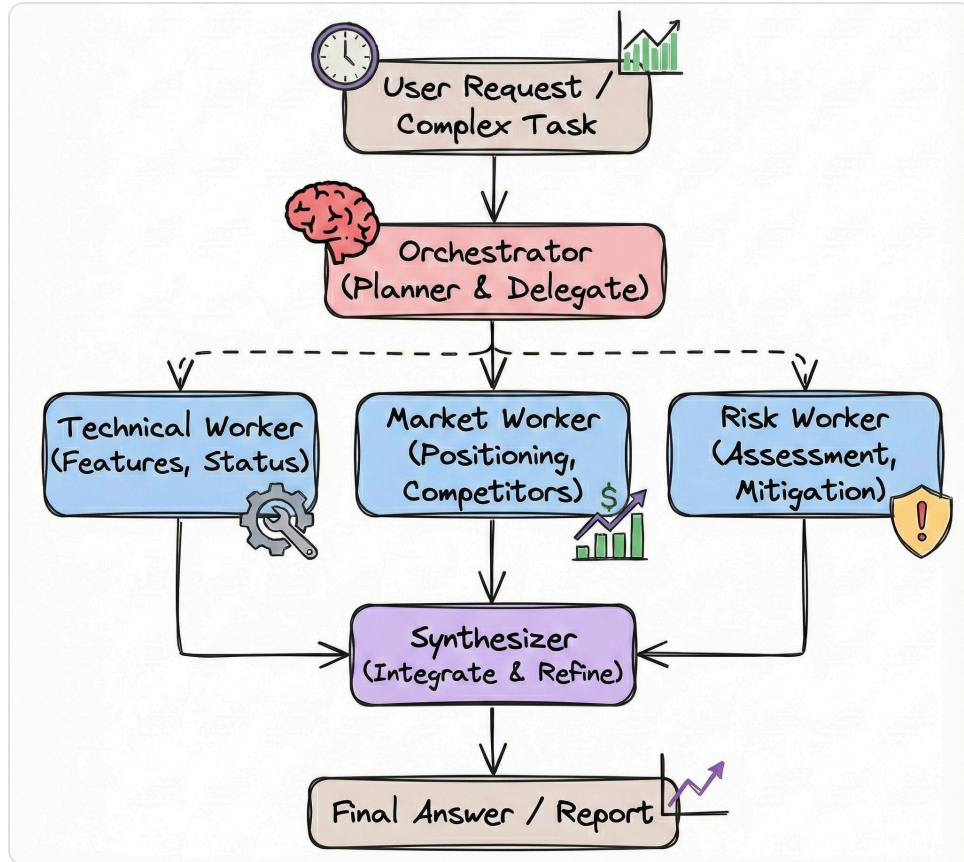
The model that produced the error is the model judging it. The same weights that made it confident the first time make it confident the second time.

SO: REFLECTION NEEDS AN EXTERNAL SIGNAL

A test that runs. A tool result. A retrieved document. A different model. A human. Something the generator did not write.

This is not an argument against reflection. It is an argument about **what the critic is allowed to be** — and it is exactly what the next slide adds.

Orchestrator and workers



THE SHAPE

A central model **breaks the task down**, delegates the pieces, and synthesises the results.

WHY IT IS NOT JUST PLAN-AND-EXECUTE

The workers run independently, often in parallel, and each gets only the slice of the problem it needs.

REACH FOR IT WHEN

The subtasks are genuinely independent, and you cannot write the decomposition in advance because it depends on the request.

The half of it that is not today's

THE QUESTIONS THIS SHAPE OPENS

- What does each worker's context window cost?
- What does a worker see, and what is hidden from it?
- What happens when two workers disagree?
- When does the whole thing stop?

ALL FOUR ARE LECTURE 08

Context isolation, token cost, hierarchies of orchestrators, shared memory and termination across agents are a lecture of their own. Today you only need to recognise the shape when you see it.

ReAct

Reasoning and acting, interleaved

First, one primitive

THE MODEL REASONS IN TEXT BEFORE IT DECIDES

It writes out what it is thinking, and **that text conditions the next choice**. Nothing more mystical than that.

WHY IT MATTERS HERE

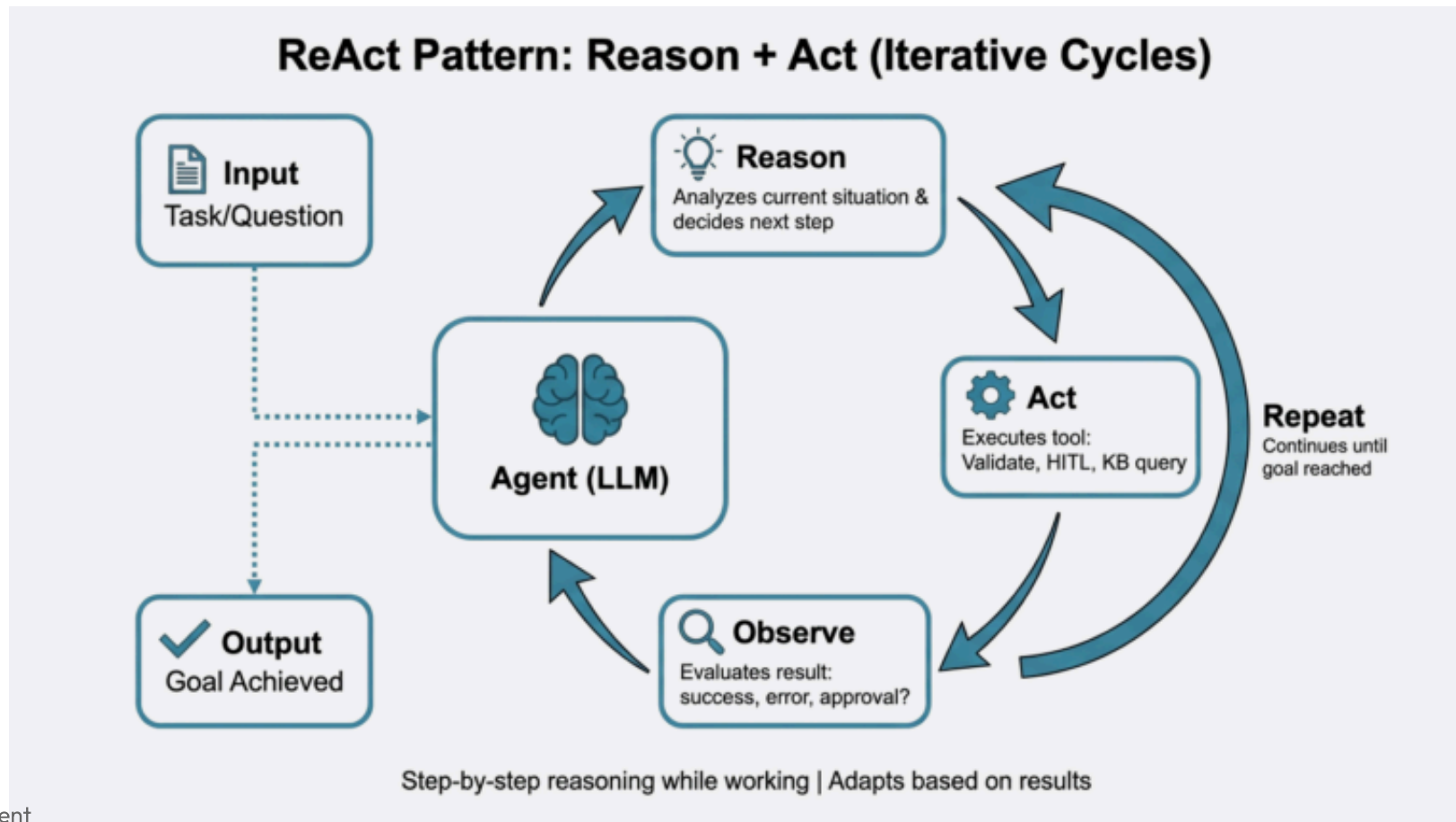
Without it, "choose a tool" is a single leap. With it, the leap is broken into visible steps you can read.

WHERE IT IS TAUGHT PROPERLY

Chain-of-thought as a *method* — variants, self-consistency, and when it makes things worse — is Lecture 10.

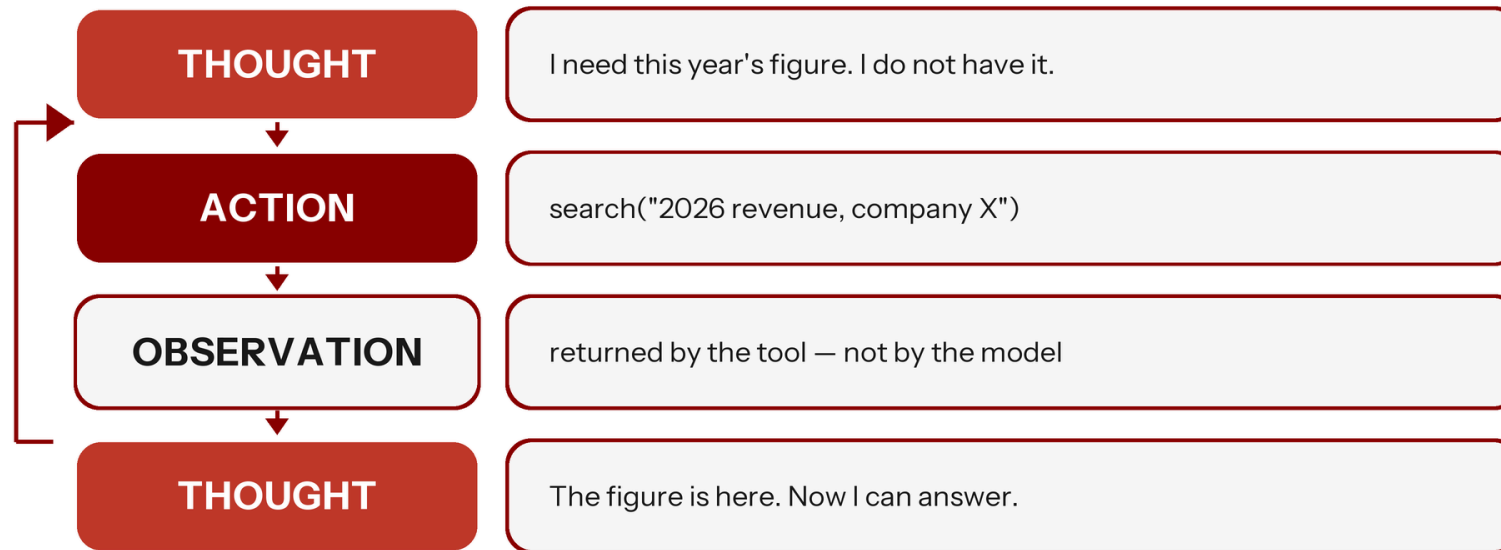
The cycle

Reason about the situation, act on it, observe what came back — then reason again with what you now know. The loop repeats until the goal is reached.



What it actually looks like

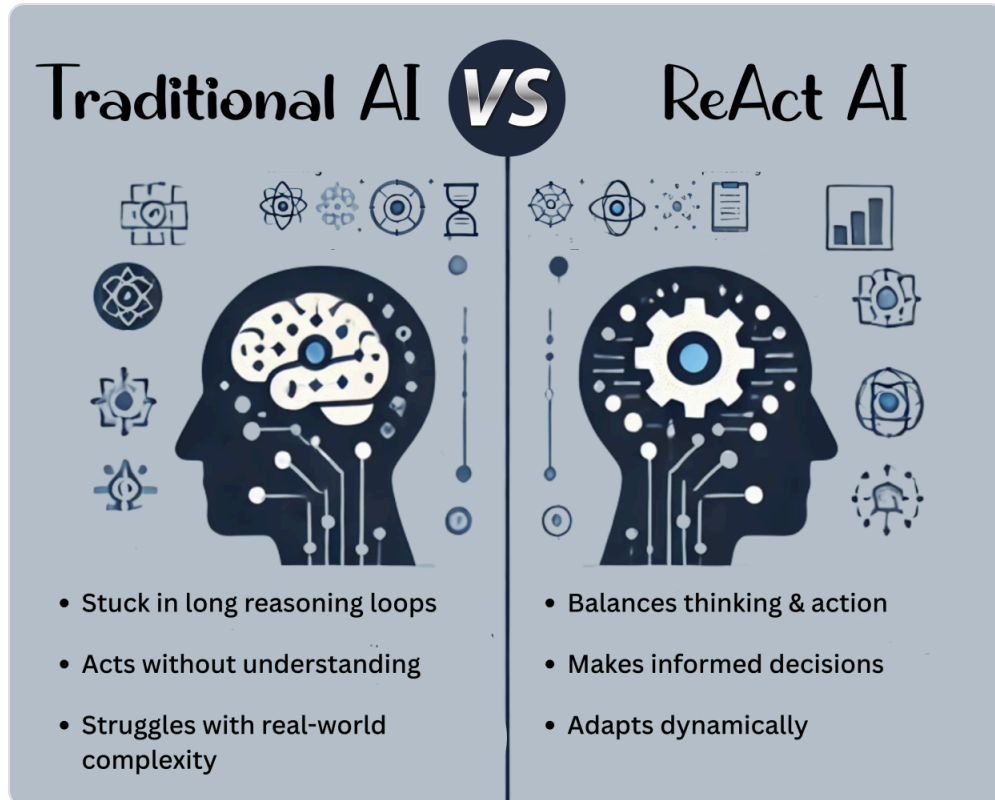
Not code — a **transcript**. The model writes the thought and the action; only the observation comes from somewhere else.



the loop repeats until the model answers instead of acting

the OBSERVATION is the environment talking back — it is what keeps the agent honest about what it actually found

Why interleaving beats planning first



REASON, THEN ACT

A plan made before any observation is a guess about a world you have not looked at yet. The first surprise invalidates the rest of it.

REASON AND ACT TOGETHER

Every observation can revise the next step. The agent is allowed to be wrong early and recover.

THE OBSERVATION IS THE IMPORTANT HALF

It is the only part of the transcript the model did not write itself.

How ReAct fails

LOOPING

The same action, repeatedly, because nothing in the observation changed the model's mind.

IGNORING THE OBSERVATION

The thought contradicts what the tool just returned — it "knows" the answer already.

NEVER TERMINATING

No condition ever says "done", so it runs until your budget does.

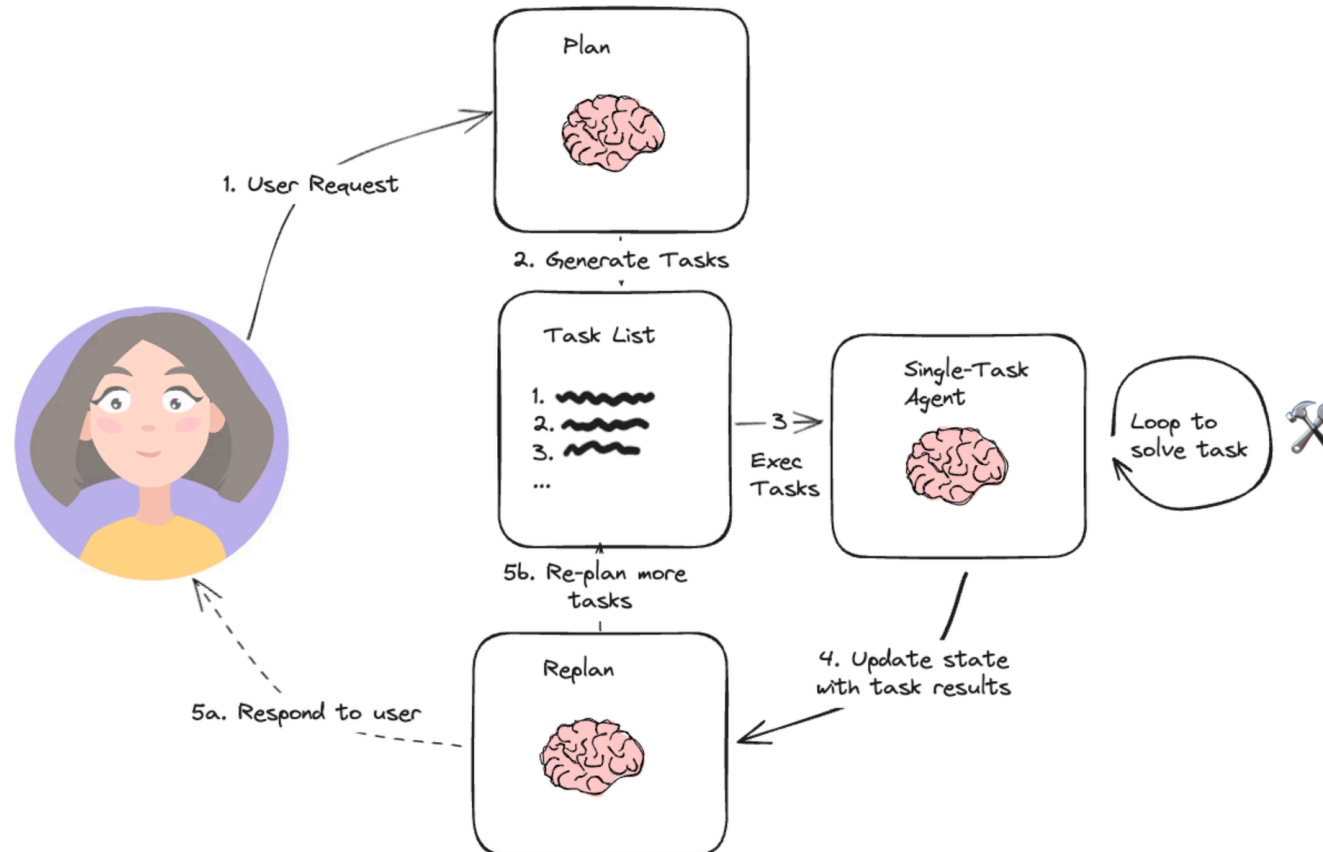
All three are why **termination** is part of the anatomy, not an afterthought.

Plan-and-Execute

Decide everything first

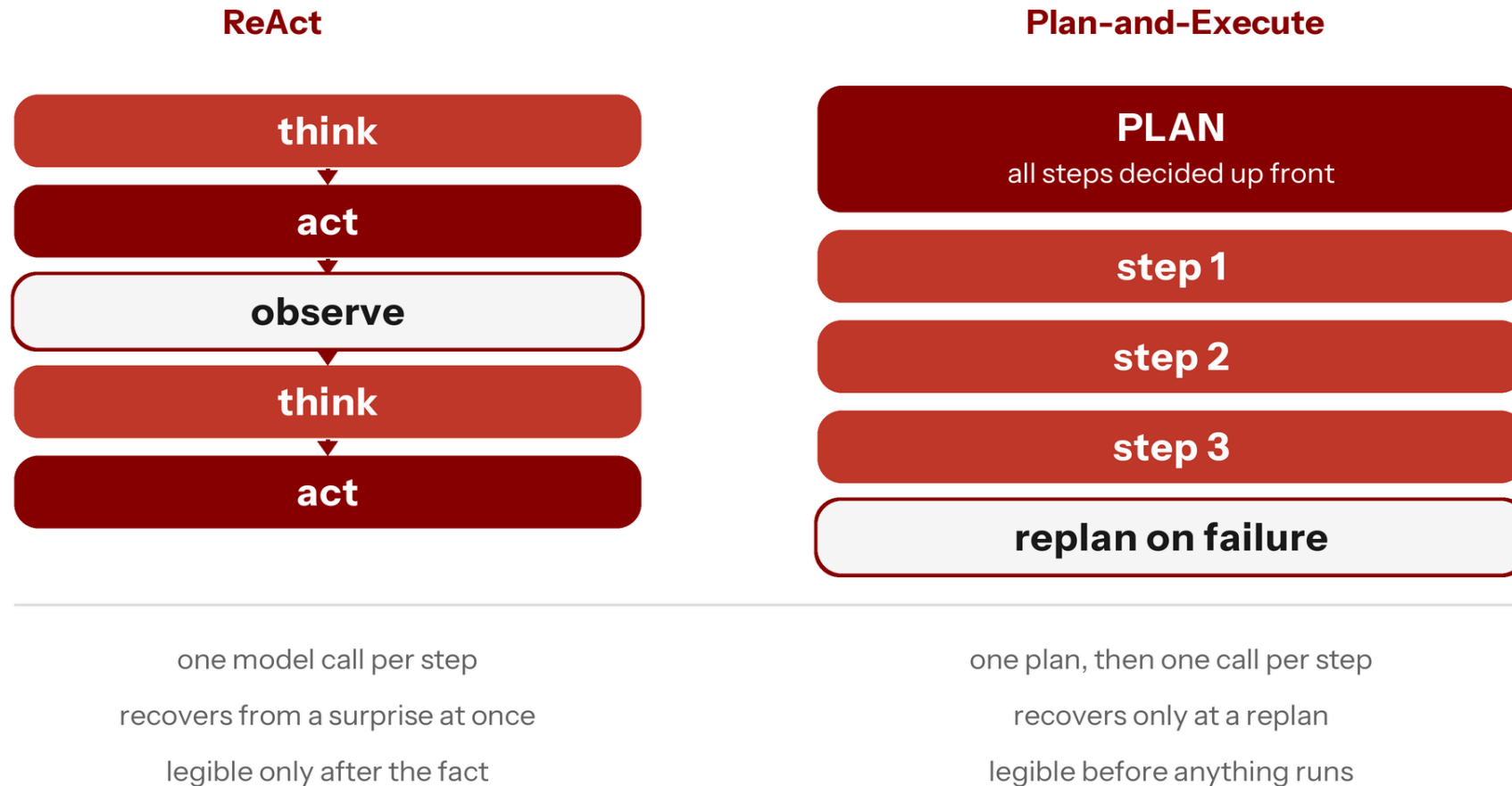
Plan, execute, replan

A planner turns the request into a task list. Each task is executed on its own. When the results come back, a replanner decides whether to answer or to add more tasks.



The contrast

Same task, two shapes — and three axes that decide which you want.



Replanning

THE PLAN WILL BE WRONG

Not because the model is bad, but because the plan was written before anything was observed. A step fails, or returns something nobody expected.

SO THE LOOP IS OUTSIDE

Execute until a step fails, then plan again with what you now know. That is the whole difference from a script.

THE TRAP

Replanning on *every* surprise turns Plan-and-Execute into an expensive ReAct. Decide what counts as a failure worth replanning for.

Choosing between them

PREFER PLAN-AND-EXECUTE

- The steps are largely predictable
- A human should approve before anything runs
- You want fewer model calls

PREFER REACT

- Each step depends on what the last one found
- Surprises are normal
- You cannot enumerate the path in advance

Neither is a default. Most real systems end up with a plan **and** a loop inside each step.

Human-in-the-Loop

An architecture decision, not an afterthought

DISCUSSION

Roughly how many steps does a deployed agent run before a human is involved?

And when an agent stops to ask you something — is that a sign it was not good enough?

Two ways to think about the human, and only one survives

TRAINING WHEELS

The agent asks because it is not good enough yet. As models improve you remove the checkpoints, and a mature system runs unattended.

Under this view, every human handoff is a defect waiting to be engineered away.

An agent that never stops for a person is not more capable. It is **unsupervised** — which is a decision someone should have made on purpose.

ARCHITECTURE

The agent asks because **the cost of being wrong about this action** exceeds the cost of asking. That is a property of the action, not of the model.

A better model does not make a wire transfer more reversible. The checkpoint stays.

Why this belongs here

THE EMPIRICAL PICTURE

Most deployed agents stop for a person within a handful of steps. Human involvement is not the fallback for a failed agent — it is the **normal operating mode**. Lecture 12 has the study.

So "where does the human go?" is a question you answer while designing, not after something goes wrong.

WHAT FOLLOWS FOR THE DESIGN

The handoff is on the main path, so it gets the same care as any other interface — not a **raise** with a log line nobody reads.

WHAT FOLLOWS FOR THE ESTIMATE

If a person is in the loop every few steps, "how many agents can one person supervise" is a capacity question, and it is the one that decides whether the system is worth building.

What to surface, and when

ALWAYS ASK BEFORE

- Anything irreversible
- Anything that spends money
- Anything that leaves your system

CONSIDER ASKING WHEN

- Confidence is low
- The cost of being wrong is high
- The action is outside a policy boundary

THE TEST THAT SETTLES MOST CASES

Ask what it costs to be wrong, and whether you can take it back. **Cheap and reversible: let it run.**
Expensive or irreversible: gate it, however confident the model sounds — confidence is not a measurement.

Designing the handoff

WHAT THE HUMAN NEEDS

- The proposed action, stated plainly
- The inputs it will use
- What happens if they say no

WHAT RUINS IT

- Asking so often they stop reading
- A prompt with no way to say "not like that"
- Approval after the action already ran

PREFER REVERSIBILITY TO APPROVAL

An action you can undo needs no gate. Design for undo first, and gate only what you genuinely cannot take back.

Next week, in code

TODAY YOU SAW

- A tool hidden behind a decorator
- An action as a line in a transcript
- A loop as an arrow going back up

NEXT WEEK YOU WRITE

- A tool as a name, description and JSON schema
- An action as a structured call your code executes
- The loop as a `while` with a termination check

Nothing new is introduced — the same five parts, in a file you can run.

Summary

COVERED

- Control against universality — and that nothing was replaced
- Agents perceive, decide and act; agents are older than LLMs
- An LLM agent is anything past a bare call — and six examples of it, in code
- Anatomy, termination, and build-or-adopt
- Six shapes, from routing to orchestrator-worker
- Human-in-the-loop as a design decision

KEEP THESE

- Every problem has its own tools — sometimes none is a model
- Never "is it an agent" — always "how far along, and why"
- A workflow runs a path you wrote; an agent writes its own
- Self-critique needs a signal from outside the model
- Termination is a feature you must write
- Flexibility you do not use is just cost

References

- Yao et al. 2023 · **ReAct: Synergizing Reasoning and Acting in Language Models** · arXiv 2210.03629
- Shinn et al. 2023 · **Reflexion: Language Agents with Verbal Reinforcement Learning** · arXiv 2303.11366 · NeurIPS 2023
- Huang et al. 2024 · **Large Language Models Cannot Self-Correct Reasoning Yet** · arXiv 2310.01798 · ICLR 2024
- Yu et al. 2025 · **Toward Super Agent System with Hybrid AI Routers** · arXiv 2504.10519 — source of the routing figure
- Russell & Norvig · **Artificial Intelligence: A Modern Approach** · ch. 2, intelligent agents
- Anthropic 2024 · **Building Effective Agents** · anthropic.com/engineering/building-effective-agents
- Dex, HumanLayer 2025 · **12-Factor Agents** · github.com/humanlayer/12-factor-agents
- LangChain · **Plan-and-Execute Agents** · langchain.com/blog/planning-agents — source of the plan-and-execute figure
- LangChain · **Reflection Agents** · langchain.com/blog/reflection-agents — source of the reflection and reflexion figures

A CLOSING QUESTION, GENUINELY OPEN

Each paradigm recovered something the last one gave up. Are agents the final answer — or is there already a trade-off forming that will produce the next shift?

What's Next · Lecture 05: Agent Development Basics

WHAT'S COVERED

- Tools, and what makes a good tool boundary
- Function calling, end to end, against raw SDKs
- Structured output, validation and retries
- Instrumentation, and what a run costs

HOW TODAY CONNECTS

- Today was the shape; next week is the wire format
- The `@tool` decorator stops being a decorator
- You will write the loop you just read

Thank You

Questions & Discussion

Lecture 04 · LLM Agent Development